rholang 1.4

MeTTaIL Language Reference

Grammar Specification

311 productions · 3 benign shift/reduce conflicts

Generated from `rholang14.cf` via BNFC 2.9.5

F1R3FLY Industries

April 8, 2026

This document reorganises the auto-generated BNFC grammar specification into thematic sections for maintainability and readability. Every production is preserved; only the presentation differs.

Contents

1 Lexical Structure **2**
- 1.1 Literal Tokens . . . 2
- 1.2 Reserved Words . . . 2
- 1.3 Symbols . . . 2
- 1.4 Comments . . . 2

2 Top-Level Program **2**

3 Processes **3**
- 3.1 Atoms (Proc16) . . . 3
- 3.2 Spatial Connectives (Proc13–15) . . . 3
- 3.3 Dereference (Proc12) . . . 3
- 3.4 Method Calls, Lambdas, and Applications (Proc11) . . . 3
- 3.5 Unary Operators (Proc10) . . . 3
- 3.6 Arithmetic (Proc8–9) . . . 3
- 3.7 Relational and Equality (Proc6–7) . . . 3
- 3.8 Boolean Connectives (Proc4–5) . . . 4
- 3.9 Send (Proc3) . . . 4
- 3.10 Binding Forms (Proc2) . . . 4
- 3.11 Conditionals, New, and Parallel (Proc0–1) . . . 4

4 Names and Channels **4**

5 For-Comprehensions and Receipts **4**
- 5.1 Receipts . . . 4
  - 5.1.1 Linear (consume) . . . 5
  - 5.1.2 Repeated (persistent listener) . . . 5
  - 5.1.3 Peek (non-consuming read) . . . 5
  - 5.1.4 MeTTaIL-Style Channel Binds . . . 5

6 Predicates (`where`-Clause Logic) **5**

7 Collections and Ground Types **6**
- 7.1 Ground Values . . . 6
- 7.2 Collections . . . 6
- 7.3 Remainders and Simple Types . . . 6

8 Patterns **6**

9 Auxiliary Productions **6**

10 GSLT Language Declarations **7**
- 10.1 Type Declarations . . . 7
- 10.2 Term Declarations . . . 7
- 10.3 Equation Declarations . . . 7
- 10.4 Rewrite Declarations . . . 8
- 10.5 Term Patterns . . . 8

11 Logic Clauses **8**

Shift/Resolve Conflict Summary **9**

Lexical Structure

Literal Tokens

Each literal class is defined by a regular expression. Position tokens carry source-location information.

Token	Regular Expression
LongLiteral^{pos}	`digit+`
FloatLiteral^{pos}	`digit+ '.' digit+ (['Ee'] '-'? digit+)?`
StringLiteral	`'"' (char−["\"] | '\' ["\\nt"])* '"'`
UriLiteral	`" ` ` " (char−["\`"] | '\' ["`\"])* " ` ` "`
Var	`(letter|`` ` ``")(letter|digit|'_'|`` ` ``")* | '_'(letter|digit|'_'|`` ` ``")+`
UpperIdent	`upper (letter|digit|'_'|`` ` ``")*`
DollarIdent	`'$' upper (letter|digit|'_'|`` ` ``")*`
RustBlock	Balanced braces, up to 3 nesting levels deep

Reserved Words

The following identifiers are reserved and cannot be used as Var:

```
Bool        ByteArray   Float    Int      Nil      Set      String    Uri
Vec         and         as       bundle   bundle0  clone    contract  else
equations   eval        false    fold     for      if       in        language
let         logic       map      match    matches  new      not       or
relation    rewrites    select   sep      terms    true     types     where
zip
```

Symbols

```
;;  {   }   ~   /\      \/      *   .   (   )   ^   [
]   -   /   %   %%      +       ++  --  <   <=  >   >=
==  !=  =   !?  |       ,       _   @   ;   <-  &   ?!
«-  ?   !   !!  bundle+ bundle- =>  #{  }#  :   ,)  ...
|-  ->  ~>  <--
```

Comments

Single-line: `//` to end of line
Multi-line: `/* ... */` (non-nesting)

Top-Level Program

A program is either a single process (rholang 1.2 backward compatibility) or a sequence of top-level declarations (MeTTaIL programs).

Program ::= *Proc* | *TopDecl**
TopDecl ::= *LangDecl* | *Proc* `;;`

Processes

Processes are stratified into 17 precedence levels, from *Proc* (level 0, lowest) to *Proc16* (atoms, highest). Braces serve as explicit grouping: `{` *Proc* `}`.

Atoms (Proc16)

Proc16 ::= *Ground* | *Collection* | *ProcVar* | `Nil` | *SimpleType* | `{}` (MeTTaIL zero process)

Spatial Connectives (Proc13–15)

Proc15 ::= `~` *Proc15* — negation
Proc14 ::= *Proc14* `/\` *Proc15* — conjunction
Proc13 ::= *Proc13* `\/` *Proc14* | *VarRefKind* Var — disjunction, var-ref

Dereference (Proc12)

Proc12 ::= `*` *Name* — eval / dereference

Method Calls, Lambdas, and Applications (Proc11)

Proc11 ::= *Proc11* `.` Var `(` *Proc** `)` — method invocation
| `(` *Proc4* `)` — parenthesised expression
| `^` Var `.` *Proc16* | `^` `[` Var$^+$ `]` `.` *Proc16* — lambda abstraction
| DollarIdent `(` *Proc** `)` — typed application
| Var `(` *Proc** `)` — function call

Unary Operators (Proc10)

Proc10 ::= `not` *Proc10* | `-` *Proc10*

Arithmetic (Proc8–9)

Proc9 ::= *Proc9* `*` *Proc10* | *Proc9* `/` *Proc10* | *Proc9* `%` *Proc10* | *Proc9* `%%` *Proc10*
Proc8 ::= *Proc8* `+` *Proc9* | *Proc8* `-` *Proc9* | *Proc8* `++` *Proc9* | *Proc8* `--` *Proc9*

Relational and Equality (Proc6–7)

Proc7 ::= *Proc7* `<` *Proc8* | *Proc7* `<=` *Proc8* | *Proc7* `>` *Proc8* | *Proc7* `>=` *Proc8*
Proc6 ::= *Proc7* `matches` *Proc7* | *Proc6* `==` *Proc7* | *Proc6* `!=` *Proc7*

Boolean Connectives (Proc4–5)

Proc5 ::= *Proc5* `and` *Proc6*
Proc4 ::= *Proc4* `or` *Proc5*

Send (Proc3)

Proc3 ::= *Name Send* `(` *Proc** `)`
Send ::= `!` | `!!`

Binding Forms (Proc2)

Proc2 ::= `contract` *Name* `(` *Name** *NameRem* `)` `=` `{` *Proc* `}`	contract
| `for` `(` *Receipt*$^{+}_{;}$ `)` `{` *Proc* `}`	for-comprehension
| `select` `{` *Branch*$^{+}$ `}`	select
| `match` *Proc4* `{` *Case*$^{+}$ `}`	match
| *Bundle* `{` *Proc* `}`	bundle
| `let` *Decl Decls* `in` `{` *Proc* `}`	let
| `(` *ChanBind*$^{+}_{,}$ `)` `.` `{` *Proc* `}`	MeTTaIL-style input

Conditionals, New, and Parallel (Proc0–1)

Proc1 ::= `if` `(` *Proc* `)` *Proc2* | `if` `(` *Proc* `)` *Proc2* `else` *Proc1*	
| `new` *NameDecl*$^{+}_{,}$ `in` *Proc1*	rholang 1.2 style
| `new` `(` Var$^{+}_{,}$ `)` `in` *Proc1*	MeTTaIL style
| *Name* `!?` `(` *Proc** `)` *SynchSendCont*	synchronous send
Proc ::= *Proc* `|` *Proc1*	parallel composition

Names and Channels

Name ::= `_` | Var | `@` *Proc12*	wildcard, variable, quote

For-Comprehensions and Receipts

The general form of a for-comprehension is:

```
  for(ptrn_{1,1} <- chan_{1,1} & ... & ptrn_{m,1} where pred_1 ;
... ;
ptrn_{1,n} <- chan_{1,n} & ... & ptrn_{m,n} where pred_n ){P}
```

Each semicolon-separated receipt-clause carries an independent set of `&`-joined binds and an optional `where`-clause. The bind counts need not be equal across clauses.

Receipts

Receipt ::= *ReceiptLinearImpl* | *ReceiptRepeatedImpl* | *ReceiptPeekImpl*

Linear (consume)

ReceiptLinearImpl ::= *LinearBind*$^{+}_{\&}$ | *LinearBind*$^{+}_{\&}$ `where` *Pred*
LinearBind ::= *Name** *NameRem* `<-` *NameSource*
NameSource ::= *Name* | *Name* `?!` | *Name* `!?` `(` *Proc** `)`

Repeated (persistent listener)

ReceiptRepeatedImpl ::= *RepeatedBind*$^{+}_{\&}$ | *RepeatedBind*$^{+}_{\&}$ `where` *Pred*
RepeatedBind ::= *Name** *NameRem* `<=` *Name*

Peek (non-consuming read)

ReceiptPeekImpl ::= *PeekBind*$^{+}_{\&}$ | *PeekBind*$^{+}_{\&}$ `where` *Pred*
PeekBind ::= *Name** *NameRem* `<<-` *Name*

MeTTaIL-Style Channel Binds

ChanBind ::= *Name* `?` Var — used in `(c?x, d?y).{P}`

Predicates (`where`-Clause Logic)

Predicates form a small logic language derived from the ascent-generated spatial and behavioural typing predicates. They are used in `where` clauses of for-comprehensions and in the `logic` section of GSLT declarations.

Disjunction uses the keyword `or` (not `;`) to avoid conflict with the receipt separator.

Pred ::= *Pred* `or` *Pred1* — disjunction
Pred1 ::= *Pred1* `,` *Pred2* — conjunction
Pred2 ::= `!` *Pred3* — negation

Atomic predicates:

Pred4 ::= Var `(` *PredArg** `)` — relational atom
| *PredArg* *CompOp* *PredArg* — comparison
| `if let` *Pattern* `=` *PredArg* — structural match
| `true` | `false` | `(` *Pred* `)`

Predicate arguments and comparison operators:

PredArg ::= Var | `_` | StringLiteral | LongLiteral | FloatLiteral | `@` *Proc12*
CompOp ::= `==` | `!=` | `>` | `<` | `>=` | `<=`

Collections and Ground Types

Ground Values

Ground ::= *BoolLiteral* | LongLiteral | FloatLiteral | StringLiteral | UriLiteral
BoolLiteral ::= `true` | `false`

Collections

Collection ::= `[` *Proc** *ProcRem* `]` — list
| *Tuple* — tuple
| `Set` `(` *Proc** *ProcRem* `)` — set
| `{` *KeyValuePair** *ProcRem* `}` — map
| `#{` *BagElem*$_{|}$ `}#` — bag / multiset (MeTTaIL)
BagElem ::= *Proc1* — | is bag separator, not par
Tuple ::= `(` *Proc* `,` `)` | `(` *Proc* `,` *Proc*$^{+}_{,}$ `)`
KeyValuePair ::= *Proc* `:` *Proc*

Remainders and Simple Types

ProcRem ::= `...` *ProcVar* | ε
NameRem ::= `...` `@` *ProcVar* | ε
SimpleType ::= `Bool` | `Int` | `Float` | `String` | `Uri` | `ByteArray`

Patterns

Patterns are used in `if` `let` guards (predicates and logic clauses) and in structural matching.

Pattern ::= `_` | Var | *Ground* | `@` *Pattern* | UpperIdent `(` *Pattern** `)` | `[` *Pattern** `]`
| `(` *Pattern* `,` *Pattern*$^{+}_{,}$ `)`

Auxiliary Productions

ProcVar ::= `_` | Var
VarRefKind ::= `=` | `=*`
Bundle ::= `bundle+` | `bundle-` | `bundle0` | `bundle`
Branch ::= *ReceiptLinearImpl* `=>` *Proc3*
Case ::= *Proc13* `=>` *Proc3*
SynchSendCont ::= `.` | `;` *Proc1*
Decl ::= *Name** *NameRem* `<-` *Proc**
Decls ::= ε | `;` *LinearDecl*$^{+}_{;}$ | `&` *ConcDecl*$^{+}_{\&}$
NameDecl ::= Var | Var `(` UriLiteral `)`

GSLT Language Declarations

The MeTTaIL extension allows defining a *graph-structured lambda theory* (GSLT) as a first-class declaration. A GSLT comprises five optional sections, each specifying a different facet of the theory.

```
LangDecl ::= language UpperIdent { LangSection* }
LangSection ::= types { TypeDecl* } | terms { TermDecl* } | equations { EqDecl* }
              | rewrites { RwDecl* } | logic { LogicClause* }
```

Type Declarations

```
TypeDecl ::= UpperIdent                                          simple category, e.g. Proc
           | ! [ TypeExpr ] as UpperIdent                        native, e.g. ![i64] as Int
           | ! [ TypeExpr ] as UpperIdent [ Str , Str , Str ]    collection with delimiters
TypeExpr ::= Var | UpperIdent | UpperIdent < TypeExpr >
```

Term Declarations

Each term declaration introduces a constructor with its typed parameters, surface syntax pattern, and target category.

```
TermDecl ::= UpperIdent . TermParams |- TermItem* : UpperIdent ;
           | UpperIdent . TermParams |- TermItem* : UpperIdent ! [ RustBlock ] fold ;
```

Parameters:

```
TermParam ::= Var : TermType                                      e.g. n:Name
            | ^ [ VarList ] . Var : [ TermBinderType ]            e.g. ^[xs].p:[Name* -> Proc]
TermType ::= UpperIdent | UpperIdent ( UpperIdent )               e.g. Vec(Name)
TermBinderType ::= UpperIdent -> UpperIdent | UpperIdent * -> UpperIdent
```

Syntax items (surface pattern to the right of `|-`):

```
TermItem ::= StringLiteral | Var | Var . * sep ( Str )            e.g. ps.*sep("|")
           | * zip ( Var , Var ) . * map ( TermLambda ) . * sep ( Str )
TermLambda ::= | VarList | TermItem*
```

Equation Declarations

Structural equivalences on terms:

```
EqDecl ::= UpperIdent . EqBinder |- TermPat = TermPat ;
EqBinder ::= ε | EqBinderItem (, EqBinderItem)*
EqBinderItem ::= Var | Var . * map ( TermLambda )
```

Rewrite Declarations

Directed reduction rules, optionally with Gillespie stochastic weights:

```
RwDecl ::= UpperIdent . |- TermPat ~> TermPat ;                                  simple
         | UpperIdent . | RwCond |- TermPat ~> TermPat ;                         conditional
         | UpperIdent . |- TermPat ~> TermPat [ Proc ] { Refinement* } ;          weighted
         | UpperIdent . | RwCond |- TermPat ~> TermPat [ Proc ] { FoldFunc } ;   context
fold
RwCond ::= Var ~> Var | Var ~> Var , RwCond
Refinement ::= Var => ( Proc , Var )
FoldFunc ::= Var
```

Term Patterns

Constructor-form patterns shared by equations and rewrites:

```
TermPat ::= TermPat1 | ( UpperIdent TermPat* )                   constructor application
          | { TermPat , ... Var }                                bag with rest
          | ( eval Var TermPat* )                                eval application
          | TermPat1 . *map ( TermPatLambda )                    map
          | ^ [ VarList ] . TermPat                              binder
TermPat1 ::= Var | UpperIdent
TermPatLambda ::= | VarList | TermPat
```

Logic Clauses

The `logic` section of a GSLT declaration defines derived relations using Datalog-style rules that compile to the `ascent` library.

```
LogicClause ::= LogicHead <-- LogicBody+, ;                      rule
              | relation Var ( LogicType+, ) ;                   relation declaration
LogicHead ::= Var ( LogicArg* )
```

Body atoms (conjunctive):

```
LogicBody ::= Var ( LogicArg* )                                  positive atom
            | ! ( Var ( LogicArg* ) )                            negated atom
            | if let Pattern = LogicArg                          structural guard
            | if LogicExpr | let Var = LogicExpr | for Var in LogicExpr
```

Expressions and arguments:

```
LogicExpr ::= Var | Var ( LogicArg* ) | LogicArg CompOp LogicArg | Var . Var ( )
LogicArg ::= Var | _ | Ground | Var . clone ( )
LogicType ::= UpperIdent | Vec < LogicType >
```

Shift/Reduce Conflict Summary

The grammar has 3 benign shift/reduce conflicts, all resolved correctly by Happy's default shift preference:

1. **`{}` ambiguity.** After `{`, the token `}` could reduce an empty *KeyValuePair* list (yielding an empty map) or shift to complete `PZero` (the MeTTaIL zero process). Shift produces `PZero`, which is the intended reading.

2. **`Var` `(` ambiguity** (two states). After `Var`, the token `(` could start the argument list of `PFuncCall` or merely follow a reduced `PVar`. Shift produces the function-call parse, which is correct: standalone `count(x,y)` should parse as a call, not a variable followed by a parenthesised expression.